

DVR/NVR FORENSIC RECOVERY PLATFORM

Master Architecture + Build Plan + Competitor-Informed Design

Software | Blockchain & Cybersecurity | NTRO | PS SIH26150

Core thesis: Build the forensic recovery engine first. Treat vendor-specific parsers as validated adapters, not as the foundation. Keep original evidence immutable; recovery, inference, and visual enhancement must remain explicitly separated.

1. Executive Summary

The proposed product is a desktop-first forensic evidence platform for DVR/NVR storage. It accepts a physical storage device or a forensic image, performs read-only acquisition and health checks, analyzes partition/filesystem structures, discovers recoverable media and metadata, attempts deterministic reconstruction where structural evidence survives, falls back to generic raw-data recovery for unknown/proprietary layouts, and uses validated vendor/format adapters when deeper metadata extraction is possible.

The architecture is deliberately inspired by proven patterns from existing forensic products: case-centric evidence handling and reporting; disk-image and filesystem analysis; modular ingest/analyzer pipelines; proprietary DVR demuxing and conversion; multi-source timeline/review; and non-destructive, auditable enhancement. These patterns should be adopted conceptually, not copied as product code or UI.

2. Competitor / Existing-Software Architecture Review

Existing products prove that the market is not waiting for a generic “DVR player.” The winning opportunity is a focused recovery engine with transparent evidence provenance and a clear recovery-confidence model.

Product / class	Observed architecture / workflow	What we should learn	Our differentiation opportunity
Magnet DVR Examiner / Magnet WITNESS	Case + source workflow; direct acquisition from surveillance DVR hard drives; recovery of deleted/partially overwritten video; multiple sources; filtering, preview, export, reports; synchronized review and sub-clipping.	Case-centric workflow, source abstraction, multiple-source review, evidence-to-report continuity.	Make recovery internals inspectable: storage health, structural findings, recovery tiers, confidence, fragment provenance, and explicit generic-vs-format-aware path.
Amped FIVE / Amped Engine	Import proprietary CCTV/DVR media; analyze codec/frame/timestamp	Separate acquisition/demux/decode from enhancement;	Make storage-level recovery + corruption repair a first-class workflow, with an

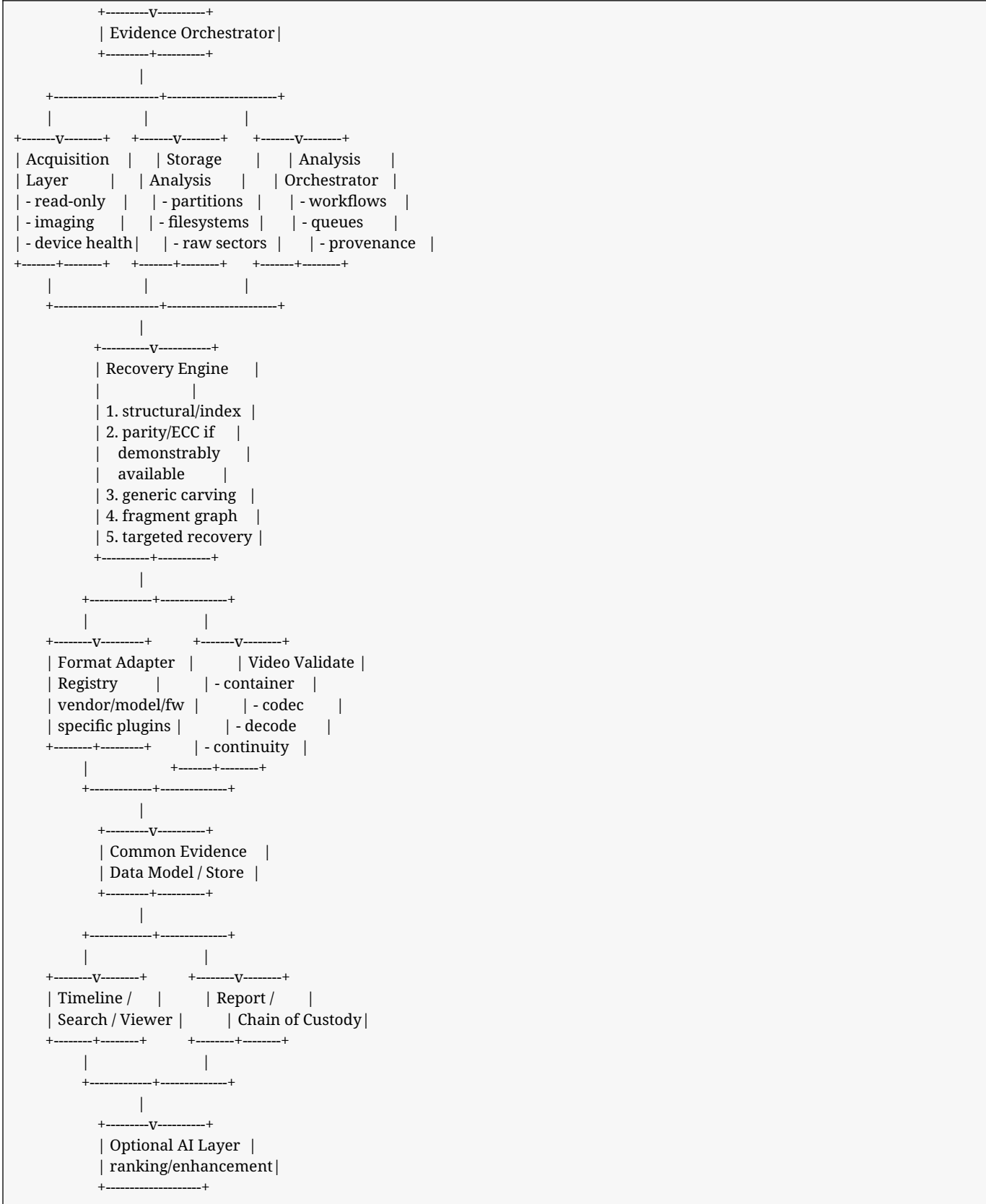
	structure; demux/extract; convert without unnecessary transcoding; restoration/enhancement; repeatable workflow and scientific report. Amped Engine is separately integrable as a CLI conversion engine.	preserve originals; record processing steps.	open modular parser registry and acquisition diagnostics.
X-Ways Forensics	Disk/image access, filesystem structures, deleted/lost partitions, RAID interpretation, sector-level access, file carving, superimposition/corrected structures.	Raw storage should remain directly inspectable; support corrupted structures without changing originals.	Specialize this forensic core for CCTV/DVR media and add video-aware recovery, timeline, and targeted clip reconstruction.
Autopsy / Sleuth Kit	Image/volume/filesystem abstraction; carving; modular analyzers; central case database; post-processing/reporting; custom modules.	Use a pipeline and artifact model; modules should communicate through a common evidence/artifact store.	Build CCTV-specific modules: DVR fingerprinting, media fragment graph, stream validation, partial-clip recovery, and provenance.
FTK Imager pattern	Write-blocked acquisition; bit-for-bit image; acquisition hashes; post-image verification.	Acquisition integrity must be deterministic and auditable.	Make acquisition health and recovery limitations visible before analysis begins.
Important: The competitor research shows that a huge feature list is not the differentiator. The differentiation must be the combination of vendor-agnostic recovery + controlled vendor adapters + transparent confidence/provenance + corruption-oriented repair workflow.			

3. What the Product Should Actually Solve

- Normal exported video that fails to open because the container/index/finalization was damaged.
- Interrupted DVR writes after power loss or recorder shutdown.
- Deleted recordings whose media bytes or metadata still remain on storage.
- Corrupted partitions/filesystem metadata where raw data is still readable.
- Partially recoverable recordings where only fragments or intervals survive.
- Unknown or unsupported vendor formats where generic recovery can still attempt to recover standard video structures.
- Targeted recovery: retrieve the relevant camera/time/event without forcing the examiner to process every recovered clip manually.
- Best-available-quality recovery when main stream data is damaged but a secondary stream, snapshot, thumbnail, or related metadata survives.
- Forensic reporting with hashes, acquisition details, recovery source, validation status, gaps, and processing history.

4. Target Architecture





5. Phase-by-Phase Build Plan with Gates

Phase 0 - Evidence lab setup

Acquire legal test media: public forensic images + at least one controlled DVR/NVR storage image. Define test corpus and expected ground truth.

Gate: Do not proceed until every test artifact has known origin, hash, expected recordings, and documented manipulation scenario. ☐ PASS ☐ FAIL

Phase 1 - Safe acquisition

Detect device, read-only mode, image creation, hashing, acquisition log, resume/error handling.

Gate: Original/source is never modified; image hash verifies; acquisition errors are logged. ☐ PASS ☐ FAIL

Phase 2 - Storage intelligence

Partition map, filesystem detection, raw-sector access, bad-sector map, device/image health report.

Gate: Known image opens; partitions are identified; unallocated regions are addressable; no writes occur. ☐ PASS ☐ FAIL

Phase 3 - Evidence discovery

File/container signatures, media stream discovery, index/journal/backup-structure discovery, metadata extraction.

Gate: At least standard-media and one DVR-specific structure are detected correctly on the test corpus. ☐ PASS ☐ FAIL

Phase 4 - Generic recovery

Carving, contiguous clip extraction, deleted-file recovery, raw-stream extraction.

Gate: A deliberately deleted/hidden test clip is recovered or the failure is explained with a deterministic report. ☐ PASS ☐ FAIL

Phase 5 - Fragment reconstruction

Build fragment database and candidate adjacency graph; use timestamps/codecs/structure as evidence; validate candidate joins by actual decoding.

Gate: Known-good synthetic fragmentation case is reconstructed without cross-camera contamination. ☐ PASS ☐ FAIL

Phase 6 - Corruption repair

Container rebuild, truncated-file recovery, metadata/index reconstruction, partial-clip extraction.

Gate: Power-loss/truncated-copy test yields playable valid segments with explicit gaps. ☐ PASS ☐ FAIL

Phase 7 - Redundancy engine

Detect and use checksums, duplicated indexes, sub-streams, and any demonstrably accessible parity/ECC/RAID redundancy.

Gate: Engine never invents parity; every reconstruction identifies its redundancy source and can be independently verified. ☐ PASS ☐ FAIL

Phase 8 - Vendor adapters

Implement one or two validated parsers for real model/firmware/format combinations.

Gate: Compatibility registry records exact validation scope; unsupported variants are clearly labeled. [] PASS [] FAIL

Phase 9 - Forensic analysis UI

Timeline, targeted search, viewer, partial clips, health report, provenance, notes, case management.

Gate: End-to-end investigator workflow works on the test corpus without manual database editing. [] PASS [] FAIL

Phase 10 - AI / enhancement

Optional fragment-ranking assistance, scene/quality analysis, denoise/deblur/super-resolution viewing copies.

Gate: AI outputs never overwrite or masquerade as source evidence; every transformation is logged. [] PASS [] FAIL

Phase 11 - Hardening

Negative tests, regression corpus, performance, crash recovery, audit-log integrity, security, packaging.

Gate: All automated tests pass; no critical data-integrity bug remains. [] PASS [] FAIL

6. Recovery Engine - Three Tiers

Tier 1 - Deterministic structural recovery

Search for primary/backup indexes, journal remnants, filesystem metadata, vendor metadata, allocation records, timestamps, and integrity fields. If these structures explicitly map recordings to storage regions, follow the mapping directly.

Evidence -> structural scan -> surviving index/metadata -> exact sector/fragment map -> reconstruct -> validate

Tier 2 - Verified fragment reconstruction

When a direct index is unavailable, create a fragment database. Candidate relationships can use channel identifiers, codec parameters, timestamps, frame ordering, neighboring structure, and other format evidence. Use a decoder as a validation gate, not as the only proof.

Fragments -> compatibility filters -> candidate edges -> score/rank -> decode-test seams -> accept/reject -> assembly

Tier 3 - Partial/degraded recovery

When complete reconstruction is impossible, preserve every independently valid fragment. Represent missing intervals in the timeline rather than silently filling the evidence with fabricated frames. Export recovered segments separately and label them clearly.

Recovered segment A -> export
Missing interval -> explicit timeline gap
Recovered segment B -> export
Original evidence remains untouched

7. Corruption Model and Repair Strategy

Failure	Likely root cause	Primary repair path	Expected result
Bad copy / download	Transport, incomplete transfer, altered bytes	Reacquire from source if available; chunk hashes; compare source vs copy	Exact corrected copy if source remains intact
Power loss during write	Incomplete container/index/finalization	Parse surviving stream; rebuild container/index; recover clean prefix/segments	Playable full or partial clip
Deleted file metadata	Directory/index entry removed, media remains	Filesystem recovery + carving + vendor index remnants	Deleted clip or fragments
Corrupted partition/filesystem	Metadata damage	Raw image analysis; reconstruct logical layout where justified; carve unallocated/unknown regions	Recovered partitions/files/streams where possible
Damaged video stream	Bad/missing sectors or bitstream corruption	Recover intact frames; isolate damaged ranges; use secondary stream for reference	Partial or degraded clip
Physical media failure	Unreadable hardware sectors/device failure	Hardware forensic acquisition/specialist; analyze whatever image is produced	Only readable image regions can be analyzed

8. Parity / ECC / Redundancy - Correct Scope

Do not assume an ordinary DVR HDD exposes video parity blocks. The software should discover redundancy, not assume it. The redundancy engine should handle: checksums/CRCs for validation; backup/duplicate indexes; secondary/sub-streams; RAID or explicit parity when the storage configuration actually provides it; and, only where the format is documented or experimentally proven, a known ECC/erasure-code reconstruction.

Redundancy discovery
 -> identify block/group mapping
 -> determine algorithm (if any)
 -> determine recoverability
 -> reconstruct missing block(s)
 -> verify against checksum/format/decoder
 -> record exact source of reconstruction

No-go rule: If there is no exposed redundancy and bits are physically absent, Hamming/XOR/Reed-Solomon cannot magically recreate the original bytes. AI may estimate visual content for a separate enhancement copy, but that is not exact evidence recovery.

9. Multi-Representation Recovery

Search the same evidence source for all surviving representations of an event: primary stream, secondary/sub-stream, snapshots, thumbnails, indexes, timestamps and cached/exported copies. If the main stream is damaged but the sub-stream survives, use it for alignment, camera/time correlation and investigative context. Do not claim that upscaling a 480p sub-stream recreates the original 1080p pixels.

Main stream 1080p -> damaged
 Sub-stream 480p -> intact
 Metadata/index -> partial

```

Snapshots    -> intact
  |
  v
correlate time/camera/event
  |
  v
recover main stream where possible
  |
  v
AI enhancement ONLY on a copied output

```

10. Common Evidence Data Model

Entity	Key fields
Case	case_id, examiner, notes, created_at
Source	source_id, type, vendor, model, firmware, capacity, hash, acquisition_method
Volume	offset, length, partition_type, filesystem, status
Fragment	fragment_id, source_id, offset_start, offset_end, stream_type, codec, channel, timestamp_start, timestamp_end, integrity_status
CandidateEdge	from_fragment, to_fragment, evidence_features, score, decode_result, decision
RecoveredArtifact	artifact_id, source_fragments, recovery_method, completeness, confidence, hash
Transformation	input_artifact, transform_type, parameters, software_version, output_artifact
AuditEvent	timestamp, actor, action, input_hash, output_hash, result, error

11. Parser / Compatibility Registry

Parser Registry Record

```

format_id: HIK-K-SERIES-FW4-X
vendor: Hikvision
model_scope: exact validated model(s)
firmware_scope: exact validated versions
storage_layout: proprietary / documented / observed
parser_version: 0.1.0
capabilities: [metadata, timestamps, camera_map]
validation_cases: [case-001, case-002]
known_limitations: [...]
status: VALIDATED | EXPERIMENTAL | UNSUPPORTED

```

Claim rule: Never write “supports Hikvision” when only one model/firmware combination has been tested. Report exact validation scope.

12. AI Placement

- Fragment-link ranking when deterministic evidence is insufficient.
- Anomaly detection on reconstructed streams.
- Quality assessment to decide which frames need enhancement.
- Search/indexing of recovered footage for objects/events.
- Denoising/deblurring/super-resolution as a separately generated viewing copy.

Deterministic evidence -> candidate fragments -> AI assistance -> decoder/format validation -> human-reviewable result

Forensic rule: AI-generated or AI-enhanced content must be labeled as inferred/enhanced. Never overwrite the original recovered artifact and never present invented pixels as the original recording.

13. Device / Hardware Failure Handling

The product should diagnose and document device accessibility, but it should not pretend to replace a hardware recovery lab. If the device does not enumerate or has controller/media damage, produce an Evidence Health Report and stop destructive operations. Continue from a specialist-produced forensic image when available.

Device detected?

YES -> read-only acquisition -> health test -> image -> verify

NO -> connection/driver diagnostics -> alternate read path if appropriate -> if still absent: hardware recovery required

14. Recommended Technology Stack

Layer	Prototype choice	Why
Desktop shell	Electron + React (or Tauri if team prefers)	Fast UI, local filesystem/device integration, cross-platform path later
Backend	Python + FastAPI	Fast iteration; good access to forensics/media libraries
Storage parsing	The Sleuth Kit / pytsk3 where applicable; libmagic / custom parsers	Volume/filesystem abstraction + custom DVR analyzers
Media	FFmpeg / ffprobe	Container/codec probing, decode validation, stream handling
Imaging	Use established forensic acquisition tooling/library first; do not reinvent low-level hardware acquisition for MVP	Reduces evidence-handling risk
Database	SQLite for local case + artifact store	Simple, portable, auditable
Hashing	SHA-256	Artifact/evidence integrity
AI	Python: PyTorch/ONNX Runtime as optional services	Keep AI isolated from source-evidence layer
Testing	Pytest + golden-image regression corpus	Every parser/recovery change must be regression-tested

15. Repository Structure

```
dvr-forensics/  
├── apps/desktop/  
├── services/api/  
├── core/  
│   ├── acquisition/  
│   ├── imaging/  
│   ├── volumes/  
│   ├── filesystems/  
│   ├── fingerprints/  
│   ├── recovery/  
│   │   ├── deterministic/  
│   │   ├── carving/  
│   │   ├── fragments/  
│   │   ├── redundancy/  
│   │   └── targeted/  
│   ├── validation/  
│   ├── provenance/  
│   ├── reports/  
└── adapters/
```

```
| ├── registry/
| ├── vendors/
| ├── ai/
| │   ├── ranking/
| │   ├── qa/
| │   └── enhancement/
| ├── tests/
| │   ├── golden_images/
| │   ├── corruption_cases/
| │   └── regression/
| └── docs/
|     ├── validation/
|     └── chain_of_custody/
```

16. Master Coding-Agent Prompt

Paste this into the coding agent you use. Force it to build phase-by-phase and stop when a gate fails.

You are the lead engineer for SIH26150, a forensic DVR/NVR evidence recovery platform. Build the product incrementally. Do NOT jump to UI polish before the recovery core works. Do NOT invent proprietary-format facts. Do NOT modify original evidence. Every phase must end with automated tests and a PASS/FAIL gate. If a gate fails, fix it before moving on.

PRODUCT PRINCIPLES

1. Read-only evidence handling; analyze forensic copies whenever possible.
2. Separate exact recovery, inferred recovery, and AI enhancement.
3. Generic recovery is the baseline; vendor/format parsers are validated plugins.
4. Never claim vendor-wide support from a single model.
5. Never fabricate missing bytes.
6. Preserve provenance for every recovered artifact.
7. Build a golden regression corpus with known ground truth.

BUILD PHASES

Phase 0: create project skeleton, test corpus manifest, threat/evidence-handling policy, and CI.

Phase 1: implement source detection, read-only checks, forensic image ingestion, SHA-256 hashing, acquisition log, and error handling.

Phase 2: implement volume/partition and filesystem inspection, raw byte reads, unallocated-region handling, and Evidence Health Report.

Phase 3: implement media fingerprinting, container/codecs probing, timestamp extraction where available, and DVR-structure discovery hooks.

Phase 4: implement generic carving and contiguous-media extraction.

Phase 5: implement fragment database + candidate adjacency graph. Use channel/codecs/metadata compatibility as filters; use timestamps and structure as evidence; require decode verification for accepted joins.

Phase 6: implement corruption repair for truncated containers, missing indexes, and partial clips. Output clean segments and explicit gaps.

Phase 7: implement Redundancy Analyzer. Detect checksums, duplicate indexes, sub-streams, RAID/parity/ECC only when the evidence exposes them. Implement mathematical reconstruction only after the redundancy layout is proven by a format adapter/test fixture.

Phase 8: implement parser registry and one real validated vendor/model/firmware adapter. Add a second only after the first is stable.

Phase 9: build case/timeline/search/viewer/reporting UI around the working core.

Phase 10: add optional AI for fragment ranking, quality assessment, search, and enhancement of copied outputs only.

Phase 11: harden security, negative tests, regression tests, performance, crash recovery, packaging.

REQUIRED STATUS VALUES

EXACT_RECOVERED, PARTIAL_RECOVERED, INFERRED, ENHANCED_COPY, UNRECOVERABLE, VALIDATED, EXPERIMENTAL, UNSUPPORTED, UNKNOWN.

FOR EVERY RECOVERY OUTPUT STORE

- source image/device ID
- source offsets/fragments
- recovery method
- validation steps

- decode result
- confidence rationale
- input/output hashes
- software/parser version
- timestamp

TEST REQUIREMENTS

Create fixtures for: normal recording, deleted recording, truncated write after power loss, corrupted container, damaged partition metadata, fragmented recording, mixed-camera fragments, missing metadata, surviving sub-stream, and insufficient-data cases. Include negative tests where the system must refuse to join incompatible fragments.

STOP CONDITIONS

- If original evidence can be modified, stop.
- If a parser claim cannot be validated, label it experimental.
- If a recovery algorithm invents bytes without a source redundancy model, stop.
- If an AI enhancement overwrites source evidence, stop.

DELIVERABLE AFTER EACH PHASE

1. Code. 2. Unit/integration tests. 3. Test output. 4. Architecture note. 5. Known limitations. 6. PASS/FAIL checklist. 7. One reproducible demo command.

FIRST TASK

Do Phase 0 only. Print the repository tree, test-corpus schema, evidence model, and PASS/FAIL checklist. Do not build the dashboard yet.

17. First 7-Day Technical Sprint

Day	Deliverable	Hard checkpoint
1	Get NIST/public forensic image + one controlled DVR image; document hashes and expected clips.	[] Images available + hashes recorded
2	Read-only acquisition/image ingestion + raw sector reader.	[] Original remains untouched
3	Partition/filesystem scan + raw-region map + health report.	[] Image fully addressable
4	Media signature scan + FFprobe/decode validation harness.	[] Known media identified
5	Controlled corruption fixtures: truncated file, deleted file, damaged container.	[] Each fixture has ground truth
6	Generic recovery + partial clip output + provenance.	[] At least one real recovery works
7	First vendor-specific sample; decide what structures are deterministic vs proprietary.	[] Go/no-go on first parser

18. Evidence / Footage Acquisition Plan

Use three kinds of test material. Do not treat ordinary online MP4 clips as DVR-disk evidence; they are useful only for codec/container and enhancement tests.

Dataset type	Use	Source / action
Public forensic disk image	Primary storage-analysis/recovery starting point	NIST CFReDS - look for the Heimvision DVR E01 forensic image.
Public CCTV/DVR sample videos	Codec/container/decode/enhancement tests only	FFmpeg sample archive includes Hikvision-labeled camera/DVR samples.
Controlled real DVR/NVR image	Core ground-truth recovery test	Borrow/buy/decommission a recorder;

		record your own footage; image the storage; perform controlled deletion/power-loss/copy-corruption scenarios.
Multiple real models	Parser validation	One or two models first; document vendor/model/firmware/storage configuration exactly.

19. What We Should Copy Conceptually - and What We Should Not

Adopt	Avoid
Case/source abstraction from forensic suites	Copying proprietary UI or product code
Modular analyzer/ingest pipeline	Claiming “supports all vendors” from a handful of devices
Raw sector + filesystem + carving layers	Assuming raw carving always reconstructs the full recording
Proprietary demux/decoder adapter pattern	Making AI the authority on evidentiary truth
Non-destructive enhancement + processing history	Overwriting recovered evidence with enhanced output
Auditable reports and hashes	Adding blockchain merely as a buzzword
Targeted filtering/timeline review	Building an enormous feature set before proving recovery

20. Critical Risks and Mitigations

Risk	Impact	Mitigation
Generic carving recovers bytes but not correct ordering	High	Fragment graph + multiple compatibility signals + decoder validation + partial output.
Vendor formats vary by model/firmware	High	Parser registry with exact validation scope.
Physical media failure	High	Stop at acquisition boundary; use hardware recovery when necessary.
AI introduces false detail	High	Separate enhancement copy; preserve source; explicit labeling.
Parity/ECC assumption proves false	High	Make redundancy discovery conditional; implement only proven algorithms/layouts.
UI built before core works	Medium	Phase gates; no Phase 9 until recovery tests pass.
Evidence altered during acquisition	Critical	Read-only controls, write blockers/established imaging path, hash verification, audit log.

21. Final Product Definition

The winning prototype is not “a player for every DVR.” It is a forensic recovery platform that starts from the storage layer, determines what evidence is actually present, extracts as much verifiable media as possible, repairs structurally damaged recordings where justified, preserves partial evidence instead of discarding it, and transparently tells the examiner how each result was obtained.

CORE USP

Vendor-agnostic evidence acquisition + deep storage analysis
+ generic media recovery
+ validated vendor/format adapters
+ corruption-aware reconstruction
+ partial evidence preservation

- + targeted recovery
- + provenance + integrity
- + optional AI assistance/enhancement

22. Research Sources Used for the Architecture Review

Magnet WITNESS product: <https://www.magnetforensics.com/products/magnet-witness/>

Magnet DVR Examiner recovery workflow: <https://www.magnetforensics.com/blog/finding-the-best-data-recovery-workflow-in-dvr-examiner-3/>

Magnet DVR Examiner data recovery: <https://www.magnetforensics.com/blog/data-recovery-with-dvr-examiner-accessible-inaccessible-and-very-inaccessible/>

Magnet WITNESS feature evolution: <https://www.magnetforensics.com/blog/introducing-magnet-witness/>

Amped FIVE: <https://ampedsoftware.com/five>

Amped Engine: <https://ampedsoftware.com/engine>

Amped proprietary-format workflow: <https://ampedsoftware.com/blog/new-feature-open-some-proprietary-dvr-or-corrupted-video-files-in-amped-five>

X-Ways Forensics: <https://x-ways.com/forensics/>

Sleuth Kit: <https://sleuthkit.org/sleuthkit/desc.php>

Sleuth Kit modular framework: <https://www.sleuthkit.org/sleuthkit/framework.php>

Autopsy: <https://www.autopsy.com/>

FTK Imager workflow reference: <https://ftkcore.com/how-it-works>

NIST CFReDS: <https://cfreds.nist.gov/all>

FFmpeg camera/DVR samples: <https://samples.ffmpeg.org/camera-dvr/hikvision/>